

UNIVERSIDAD INDOAMÉRICA

FACULTAD DE INGENIERÍAS

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

TEMA:

MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-8B EN NVIDIA A100 MEDIANTE VLLM

Trabajo de Integración Curricular previo a la obtención del título de Ingeniero en Tecnologías de la Información

Autor

José Andrés Ureta Granda

Tutor

Andrés Xavier Rubio Proaño

QUITO – ECUADOR

2026

AUTORIZACIÓN POR PARTE DEL AUTOR PARA LA CONSULTA, REPRODUCCIÓN PARCIAL O TOTAL, Y PUBLICACIÓN ELECTRÓNICA DEL TRABAJO DE INTEGRACIÓN CURRICULAR

Yo, José Andrés Ureta Granda, declaro ser autor del Trabajo de Titulación con el nombre "Modelado experimental de las fases prefill y decode de Qwen3-8B en NVIDIA A100 mediante vLLM", como requisito para optar al grado de Ingeniero en Tecnologías de la Información y autorizo al Sistema de Bibliotecas de la Universidad Tecnológica Indoamérica para que, con fines netamente académicos, divulgue esta obra a través del Repositorio Digital Institucional (RDI-UTI).

Los usuarios del RDI-UTI podrán consultar el contenido de este trabajo en las redes de información del país y del exterior, con las cuales la Universidad tenga convenios. La Universidad Tecnológica Indoamérica no se hace responsable por el plagio o copia del contenido parcial o total de este trabajo.

Del mismo modo, acepto que los Derechos de Autor, Morales y Patrimoniales sobre esta obra serán compartidos entre mi persona y la Universidad Tecnológica Indoamérica, y que no tramitaré la publicación de este trabajo en ningún otro medio sin autorización expresa de la misma. En caso de que exista el potencial de generación de beneficios económicos o patentes, producto de este trabajo, acepto que se deberán firmar convenios específicos adicionales donde se acuerden los términos de adjudicación de dichos beneficios.

Para constancia de esta autorización, en Quito, Ecuador, a los xx días del mes de xx de 2026, firmo conforme:

Autor: José Andrés Ureta Granda

Firma:

Número de Cédula: 1751545185
Dirección: Pichincha, Quito, Belisario Quevedo, La Gasca
Correo Electrónico: andredks334@gmail.com
Teléfono: 0939116995

APROBACIÓN DEL TUTOR

En mi calidad de Tutor del Trabajo de Titulación "MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-8B EN NVIDIA A100 MEDIANTE VLLM", presentado por José Andrés Ureta Granda, para optar por el Título de Ingeniero en Tecnologías de la Información.

CERTIFICO

Que dicho Trabajo de Titulación ha sido revisado en todas sus partes y considero que reúne los requisitos y méritos suficientes para ser sometido a la presentación pública y evaluación por parte del Tribunal revisor que se designe.

Quito, Ecuador, xx de xx de 2026

Andrés Xavier Rubio Proaño

DECLARACIÓN DE AUTENTICIDAD

Yo, José Andrés Ureta Granda, declaro que los contenidos y los resultados obtenidos en el presente Trabajo de Titulación "Modelado experimental de las fases prefill y decode de Qwen3-8B en NVIDIA A100 mediante vLLM", como requerimiento previo para la obtención del Título de Ingeniero en Tecnologías de la Información, son absolutamente originales, auténticos y personales y de exclusiva responsabilidad legal y académica del autor.

Quito, Ecuador, xx de xx de 2026

José Andrés Ureta Granda
(Número de cédula)

APROBACIÓN DE LECTORES

El Trabajo de Titulación ha sido revisado, aprobado y autorizada su impresión y empastado sobre el tema "MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-8B EN NVIDIA A100 MEDIANTE VLLM", presentado por José Andrés Ureta Granda, previo a la obtención del Título de Ingeniero en Tecnologías de la Información, y reúne los requisitos de fondo y forma para su sustentación.

Quito, Ecuador, xx de xx de 2026

Nombres Apellidos
LECTOR

Nombres Apellidos
LECTORA

DEDICATORIA

(Obligatoria)
La dedicatoria va en la parte inferior derecha de la página.

AGRADECIMIENTO

(Obligatorio)
El agradecimiento va en la parte inferior derecha de la página.

Índice de Contenidos

CAPÍTULO I: Introducción **16**
1.1 Antecedentes 17
1.1.1 Contexto 17
1.1.2 Estado del arte 18
1.2 Planteamiento del problema 20
1.3 Formulación del problema 21
1.4 Objetivos 21
1.4.1 Objetivo general 21
1.4.2 Objetivos específicos 22
1.5 Justificación 23
1.6 Alcance y delimitaciones 24
1.6.1 Alcance 25
1.6.2 Delimitaciones 25
1.7 Fundamentación teórica 26

CAPÍTULO II: Metodología **27**
2.1 Diagnóstico de la Situación Actual 27
2.2 Área de Estudio 27
2.3 Metodología de Desarrollo o Implementación 28
2.4 Técnicas e Instrumentos de Recolección de Información 28
2.5 Requerimientos del Proyecto 28
2.5.1 Requerimientos Funcionales 28
2.5.2 Requerimientos No Funcionales 28
2.6 Diseño de la Solución 28
2.7 Justificación Técnica 28
2.7.1 Ejemplo de Imagen 1 29
2.7.2 Ejemplo de Ecuación 1 29

CAPÍTULO III: Implementación, Pruebas y Resultados **30**
3.1 Descripción de la Solución 30

3.2 Proceso de Implementación 30
3.3 Pruebas y Validación 30
3.4 Resultados 30
3.4.1 Ejemplo de Figura 2 30
3.4.2 Ejemplo de Tabla 2 31
3.4.3 Ejemplo de Imagen 2 31
3.4.4 Ejemplo de Ecuación 2 31
3.4.5 Ejemplos de Referencias Cruzadas 32
3.5 Cronograma y Presupuesto 32

CAPÍTULO IV: Conclusiones y Recomendaciones **33**
4.1 Conclusiones 33
4.2 Recomendaciones 33
4.3 Trabajos Futuros 33

REFERENCIAS **34**

GLOSARIO **37**

ANEXO A: Título del Anexo A **38**

Índice de Tablas

1 Comparación de los trabajos revisados en el estado del arte. 19

2 Área de estudio. 27

3 Ejemplo de tabla comparativa antes y después de la implementación. 31

Índice de Figuras

1 Ejemplo de figura para presentar resultados o arquitectura implementada. 31

Índice de Imágenes

1 Ejemplo de imagen para documentación visual de infraestructura o evidencias. 29

2 Ejemplo de imagen para evidenciar interfaz, panel o consola de operación. 32

Índice de Ecuaciones

1 Cálculo de disponibilidad del servicio 29
2 Cálculo porcentual de mejora obtenida 32

Índice de Anexos

Anexo A: Título del Anexo A 38

UNIVERSIDAD INDOAMÉRICA

FACULTAD DE INGENIERÍAS

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

TEMA: MODELADO EXPERIMENTAL DE LAS FASES PREFILL Y DECODE DE QWEN3-8B EN NVIDIA A100 MEDIANTE VLLM

AUTOR(A): José Andrés Ureta Granda

TUTOR(A): Andrés Xavier Rubio Proaño

RESUMEN EJECUTIVO

[Redacte el resumen del Trabajo de Integración Curricular en un solo párrafo, con un máximo de 300 palabras. Exponga de manera clara el problema, el objetivo general, la metodología empleada, los principales resultados obtenidos y las conclusiones.]

DESCRIPTORES: *[Incluya de 3 a 4 palabras clave representativas, ordenadas alfabéticamente y separadas por comas.]*

UNIVERSIDAD INDOAMÉRICA

FACULTY OF ENGINEERING

BACHELOR'S DEGREE IN INFORMATION TECHNOLOGY ENGINEERING

TOPIC: EXPERIMENTAL MODELING OF THE PREFILL AND DECODE PHASES OF QWEN3-8B ON NVIDIA A100 USING VLLM

AUTHOR: José Andrés Ureta Granda

ADVISOR: Andrés Xavier Rubio Proaño

ABSTRACT

[Write the English version of the executive summary in a single paragraph, with a maximum of 300 words. Clearly present the problem, general objective, methodology, main results, and conclusions.]

KEYWORDS: *[Include 3 to 4 representative keywords in alphabetical order, separated by commas.]*

CAPÍTULO I

INTRODUCCIÓN

La puesta en servicio de un modelo de lenguaje de gran tamaño (LLM, por sus siglas en inglés) obliga a elegir una configuración de inferencia, y esa elección suele apoyarse en métricas agregadas como los tokens por segundo. Estas métricas resumen en una sola cifra dos fases de ejecución, prefill y decode, que someten a la unidad de procesamiento gráfico (GPU) a exigencias distintas [1]. Una cifra agregada registra, por tanto, el resultado de una configuración sin revelar en qué fase se gana o se pierde rendimiento.

El caso de Qwen3-8B [2] sobre una GPU NVIDIA A100 [3], servido mediante el motor de inferencia vLLM [4], vuelve concreta esta limitación. La ficha institucional del trabajo constata que el modelo puede ejecutarse sin dificultad en una A100 y que, aun así, ninguna configuración resulta óptima para todas las cargas de trabajo. Si la configuración adecuada depende de la carga, elegirla exige conocer cómo responde cada fase a los factores que la definen.

De esta tensión surge la pregunta central del capítulo, que la Sección 1.3 formula de forma explícita: cómo influyen la cuantización, la longitud de contexto y la concurrencia en el comportamiento de las fases prefill y decode de Qwen3-8B, ejecutado mediante vLLM sobre una GPU NVIDIA A100, en términos de rendimiento, memoria, energía y calidad. En esta pregunta, la concurrencia designa el número de solicitudes simultáneas. La longitud de contexto comprende la longitud de entrada, o número de tokens del texto de entrada, y la longitud de salida, o número de tokens generados por solicitud.

El recorrido del capítulo sigue el orden en que se construye esa pregunta y su respuesta. La Sección 1.1 reúne el contexto del trabajo y el estado del arte para localizar el vacío que la literatura deja abierto; la Sección 1.2 expone las consecuencias de ese vacío para el despliegue. La Sección 1.3 convierte esas consecuencias en la pregunta principal, y la Sección 1.4 la descompone en una cadena de objetivos en la que cada uno habilita al siguiente. La Sección 1.5 justifica por qué la pregunta merece respuesta y la Sección 1.6 fija el dominio en que esa respuesta será válida. La Sección 1.7 cierra el capítulo con los fundamentos teóricos que el resto de la memoria requiere.

1.1 Antecedentes

Formular la pregunta central requiere situar antes el trabajo en dos planos: el contexto técnico e institucional del que surge y los aportes previos sobre los que se apoya. La Sección 1.1.1 describe el primero y la Sección 1.1.2 revisa los segundos.

1.1.1 Contexto

Los LLM se han incorporado a asistentes conversacionales, sistemas de consulta documental y flujos de procesamiento automático de texto. Estos modelos se construyen, en su mayoría, sobre la arquitectura Transformer [5]. Una vez entrenado, cada modelo se ejecuta en modo de inferencia ante cada solicitud recibida, de modo que el costo de servirlo se acumula durante toda su vida operativa. La eficiencia de la inferencia condiciona, en consecuencia, la viabilidad de cualquier despliegue.

La inferencia de un LLM se desarrolla en las dos fases ya mencionadas, prefill y decode [1]. La fase prefill procesa el texto de entrada de la solicitud, o prompt, y produce el primer token de salida; la fase decode genera los tokens restantes de uno en uno, de forma autorregresiva. Durante ambas fases, el modelo conserva en la memoria de la GPU las claves y los valores de atención de los tokens previos, estructura que se denomina KV cache. Su funcionamiento se desarrolla en la Sección 1.7; aquí basta señalar que la memoria que ocupa crece con la longitud de contexto y con el número de solicitudes atendidas [4].

Cada fase somete a la GPU a exigencias diferentes, y la configuración de inferencia determina cómo se reparten los recursos entre ambas [1], [6]. En este trabajo, la configuración de inferencia designa la combinación de representación numérica del modelo y parámetros del motor de inferencia con que se atiende una carga de trabajo. De ese reparto dependen dos magnitudes que interesan a quien despliega el servicio: la latencia percibida por los usuarios y el throughput, entendido como el número de tokens procesados por segundo. Una configuración que favorece una de ellas puede perjudicar la otra [6]; de ahí que la distinción entre fases ocupe un lugar central en los trabajos revisados en la Sección 1.1.2.

El presente trabajo de fin de grado (TFG) se enmarca en el proyecto LLM-CoDesign, que caracteriza la inferencia de modelos de lenguaje considerando conjuntamente el modelo, la carga de trabajo, el hardware y el software. Según la ficha institucional del TFG, el proyecto abarca múltiples modelos, GPU, motores de inferencia y métricas durante cinco años. Un alcance de esa amplitud necesita un primer caso controlado sobre el cual validar la metodología antes de extenderla. Este TFG reduce el alcance del proyecto a ese caso y constituye su primer caso de estudio.

El caso seleccionado combina el modelo Qwen3-8B, perteneciente a la familia Qwen3 [2], con una GPU NVIDIA A100 y el motor de inferencia vLLM [4]. La A100 se comercializa en variantes de 40 GB y 80 GB de memoria [3]. Fijado el caso, queda por establecer qué sabe ya la literatura sobre la inferencia de LLM en una GPU y qué deja sin resolver para esta combinación.

1.1.2 Estado del arte

La distinción entre prefill y decode, presentada en la sección anterior, atraviesa la literatura sobre inferencia de LLM. Esa literatura se concentra en mecanismos de sistema orientados a aprovechar mejor la GPU: la planificación de solicitudes, la gestión de memoria, los kernels de atención, el tratamiento diferenciado de las fases y la cuantización. Esta sección revisa seis trabajos representativos de esas líneas y señala, en cada caso, qué aportan a la pregunta central y qué dejan abierto.

Yu et al. [7] proponen Orca, un motor de inferencia que planifica la ejecución a nivel de iteración en lugar de a nivel de solicitud. Con esta estrategia, el sistema incorpora solicitudes nuevas al lote y retira las finalizadas en cada paso de generación. Orca reserva, sin embargo, el espacio del KV cache de cada solicitud en bloques contiguos dimensionados de antemano, y esa reserva desperdicia una parte de la memoria disponible [4]. La planificación por iteración mejora así el aprovechamiento del cómputo, mientras deja abierto el de la memoria.

Kwon et al. [4] abordan ese problema con PagedAttention, un mecanismo que gestiona el KV cache en bloques de tamaño fijo, de forma análoga a la memoria virtual paginada de un sistema operativo. La gestión paginada reduce la fragmentación de la memoria y permite compartir bloques entre solicitudes. Sobre este mecanismo, los autores construyen vLLM, que combina la gestión paginada con la planificación a nivel de iteración, conocida como batching continuo. vLLM es el motor de inferencia empleado en este trabajo; su evaluación original, sin embargo, no separa el comportamiento de prefill y decode.

Dao et al. [8] desarrollan FlashAttention, un algoritmo de atención exacta que reduce los accesos a la memoria de alto ancho de banda de la GPU mediante el procesamiento por bloques. El trabajo pone de relieve el peso del movimiento de datos entre niveles de memoria en el costo de la atención. Esta perspectiva resulta pertinente para el presente TFG, porque el límite que impone la memoria frente al que impone el cómputo es precisamente lo que distingue a una fase de la otra [1].

Agrawal et al. [6] estudian el compromiso entre throughput y latencia y observan que la incorporación del prefill de solicitudes nuevas a un lote interrumpe la generación de las solicitudes que se encuentran en decode. Como respuesta, proponen

Sarathi-Serve, un motor de inferencia que divide el prefill en fragmentos (chunked prefill) y los planifica sin detener la generación en curso. La coexistencia de ambas fases en un mismo lote tiene una consecuencia directa para este trabajo: condiciona la manera de medirlas por separado.

Zhong et al. [1] llevan la separación más lejos con DistServe, un motor de inferencia que asigna prefill y decode a GPU distintas para eliminar la interferencia entre fases. Su análisis caracteriza prefill como una fase limitada por el cómputo y decode como una fase limitada por el ancho de banda de memoria. Los autores evalúan cada fase con una métrica propia: el TTFT (*time to first token*), tiempo hasta el primer token, asociado a prefill; y el TPOT (*time per output token*), tiempo por token de salida, asociado a decode. Este trabajo adopta ambas métricas, aunque su caso de una sola GPU no admite la separación física que DistServe propone.

Lin et al. [9] proponen AWQ, un método de cuantización de solo pesos a baja precisión que protege los pesos más relevantes a partir de la distribución de las activaciones. La representación de los pesos con 4 bits (AWQ INT4) reduce la memoria ocupada por el modelo frente al formato de coma flotante de 16 bits BF16. La memoria que libera la cuantización queda disponible para el KV cache, de modo que la representación numérica puede alterar el comportamiento de ambas fases. Por esa razón, este TFG compara las dos representaciones numéricas.

La Tabla 1 resume el aporte, la limitación principal y la relación de cada trabajo con el presente TFG. Los seis trabajos proponen o evalúan mecanismos de sistema, y ninguno caracteriza la combinación de Qwen3-8B, A100 y vLLM. Todos ellos son, además, anteriores a la publicación de la familia Qwen3 [2].

Tabla 1: Comparación de los trabajos revisados en el estado del arte.

Trabajo	**Aporte principal**	**Limitación principal**	**Relación con el TFG**
Orca [7]	Planificación a nivel de iteración	Reserva contigua del KV cache	Origen del batching continuo de vLLM
vLLM [4]	Gestión paginada del KV cache	No separa el análisis por fases	Motor de inferencia del TFG
FlashAttention [8]	Atención exacta con menos accesos a memoria	Optimiza un kernel aislado	Interpretación de cuellos de botella
Sarathi-Serve [6]	Chunked prefill sin detener decode	Mezcla ambas fases en un mismo lote	Condiciona la medición por fases
DistServe [1]	Separación de prefill y decode en GPU distintas	Requiere varias GPU	Define TTFT y TPOT por fase
AWQ [9]	Cuantización de pesos a 4 bits	Posible degradación de calidad	Representación AWQ INT4 evaluada

Elaborado por: José Andrés Ureta Granda
Fuente: Elaboración propia

La literatura revisada explica, por tanto, por qué prefill y decode se comportan de forma distinta, pero no ofrece una caracterización experimental de ambas fases para el caso seleccionado. La Sección 1.2 examina las consecuencias de ese vacío para quien debe configurar el servicio.

1.2 Planteamiento del problema

El vacío identificado en el estado del arte tiene un efecto práctico. La ficha institucional del trabajo constata que la selección de una configuración de inferencia suele basarse en métricas agregadas, como los tokens por segundo, sin distinguir el comportamiento de las fases prefill y decode. Una métrica agregada resume en una sola cifra dos procesos con exigencias distintas sobre la GPU [1]. El resultado es una visión incompleta del sistema, que registra los compromisos entre fases sin explicarlos.

Esa visión incompleta deja sin explicar varios fenómenos relevantes para el despliegue. No permite justificar por qué una configuración mejora el throughput y empeora la latencia interactiva, ni en qué casos la cuantización permite aumentar la concurrencia. Tampoco aclara cómo influye la longitud de entrada sobre el TTFT, ni cómo afecta el crecimiento del KV cache a la latencia durante decode. Cada uno de estos fenómenos se origina en una fase concreta; por ello, ninguna cifra agregada alcanza a explicarlo.

La consecuencia práctica es la dificultad para elegir una configuración adecuada según el uso previsto. Según la ficha institucional, un servicio de chat interactivo, una consulta documental basada en recuperación (RAG, por sus siglas en inglés) y un procesamiento por lotes imponen prioridades diferentes. Una configuración apropiada para un escenario puede resultar, por tanto, inadecuada para otro.

El caso de Qwen3-8B en una A100 ilustra la situación. La ficha institucional constata que el modelo puede ejecutarse sin dificultad en esa GPU, pero esta posibilidad no garantiza la existencia de una configuración universalmente óptima: el rendimiento depende de la carga de trabajo, de la concurrencia y de la representación numérica del modelo. Para esta combinación concreta no se dispone, además, de una caracterización experimental que separe ambas fases, como muestra la Tabla 1. Ese vacío, trasladado al despliegue, constituye el problema que la Sección 1.3 convierte en pregunta.

1.3 Formulación del problema

El problema descrito en la sección anterior se formula mediante la siguiente pregunta principal: ¿cómo influyen la cuantización, la longitud de contexto y la concurrencia en el comportamiento de las fases prefill y decode de Qwen3-8B, ejecutado mediante vLLM sobre una GPU NVIDIA A100, en términos de rendimiento, memoria, energía y calidad?

La pregunta principal se descompone en preguntas específicas que la vuelven medible. Las dos primeras atañen a cada fase por separado: cómo cambia el TTFT cuando aumenta la longitud de entrada y cómo cambia el TPOT cuando aumenta la concurrencia. La tercera examina cómo influye la longitud de salida en la latencia y en la energía.

Otras tres preguntas se ocupan de la memoria y de la capacidad del sistema: cuánta memoria ocupan las representaciones numéricas BF16 y AWQ INT4, en qué condiciones la cuantización mejora la capacidad del sistema y qué configuraciones producen saturación o errores por falta de memoria. Las dos últimas cierran el recorrido: cuáles son los principales cuellos de botella de cada fase y con qué error estima un modelo predictivo básico el TTFT y el TPOT a partir de las variables experimentales delimitadas en la Sección 1.6.1.

Responder a estas preguntas exige una secuencia de productos: un entorno en el que ejecutar el modelo, una carga controlada, un conjunto de mediciones, un análisis por fases y un modelo predictivo. La Sección 1.4 organiza esos productos como una cadena de objetivos.

1.4 Objetivos

Las preguntas de la sección anterior se responden mediante un objetivo general y cinco objetivos específicos. Sus enunciados reproducen literalmente los de la ficha institucional aprobada.

1.4.1 Objetivo general

Caracterizar y modelar experimentalmente el comportamiento de las fases prefill y decode de Qwen3-8B sobre una GPU NVIDIA A100, evaluando el efecto de la cuantización, la longitud de contexto y la concurrencia sobre el rendimiento, la memoria, la energía y la calidad.

El objetivo general traslada la pregunta principal a una acción verificable. Retoma los mismos tres factores y las mismas cuatro dimensiones que la pregunta nombra,

y distingue dos tareas: caracterizar el comportamiento de cada fase y modelarlo.

1.4.2 Objetivos específicos

OE1. Implementar un entorno reproducible para ejecutar Qwen3-8B mediante vLLM sobre una A100.

OE2. Diseñar un benchmark parametrizable para generar cargas con diferentes longitudes de entrada, salida y niveles de concurrencia.

OE3. Medir TTFT, TPOT, throughput, memoria GPU, utilización y consumo energético.

OE4. Analizar separadamente las fases prefill y decode e identificar sus principales cuellos de botella.

OE5. Construir un modelo predictivo básico para estimar TTFT y TPOT.

Los cinco objetivos específicos se leen como una cadena y no como una lista. Los párrafos siguientes explican, para cada uno, qué vacío cierra, por qué condiciona al objetivo que le sigue y qué aporta al objetivo general.

OE1 cierra la ausencia de un entorno controlado en el que medir. La ficha institucional exige congelar y documentar las versiones de CUDA, del controlador de NVIDIA, de PyTorch y de vLLM, de modo que los experimentos puedan reconstruirse. Sin esa condición, ninguna diferencia observada entre configuraciones podría atribuirse a los factores estudiados en lugar de a cambios del entorno. Por esa razón, OE1 es condición necesaria de OE2: un benchmark solo produce cargas comparables si se ejecuta siempre sobre el mismo entorno. Al objetivo general, OE1 le aporta la reproducibilidad de la caracterización.

OE2 cierra la distancia entre la pregunta y los datos. Las preguntas específicas varían la longitud de entrada, la longitud de salida y la concurrencia, y ningún entorno genera por sí solo cargas con esos niveles controlados. El benchmark parametrizable produce esas cargas de forma sistemática sobre el entorno de OE1. Es condición necesaria de OE3, porque una métrica solo puede atribuirse a un factor si la carga que la produjo está controlada. Al objetivo general, OE2 le aporta la variación sistemática de la longitud de contexto y de la concurrencia, mientras que la cuantización se introduce al ejecutar el modelo en sus dos representaciones numéricas.

OE3 cierra la ausencia de mediciones para esta combinación, señalada en la Tabla 1. El TTFT y el TPOT reflejan cada fase por separado; el throughput, la memoria GPU, la utilización y el consumo energético cubren las dimensiones de

rendimiento, memoria y energía del objetivo general. Es condición necesaria de OE4, porque el análisis por fases requiere datos medidos bajo las cargas controladas de OE2. El consumo energético se obtiene como estimación a partir de la telemetría interna de la GPU, según precisa la Sección 1.6.1.

OE4 cierra el vacío central del planteamiento: la explicación de los compromisos que las métricas agregadas registran sin explicar. Separar prefill y decode permite atribuir cada efecto observado a la fase que lo produce y localizar en cuál de ellas aparece cada cuello de botella. Es condición necesaria de OE5, porque el modelo predictivo estima el TTFT y el TPOT por separado, y cada estimación exige conocer qué variables gobiernan la fase correspondiente. Con OE4 se completa la parte de caracterización del objetivo general.

OE5 cierra el paso de la descripción a la anticipación. Un modelo predictivo básico estima el TTFT y el TPOT de una configuración a partir de las variables experimentales, sin necesidad de ejecutarla, dentro de los niveles estudiados. Como último eslabón, OE5 no habilita otro objetivo específico; alimenta, en cambio, la estimación que realiza el prototipo Qwen-A100 Profiler descrito en la Sección 1.5. Con él se completa la parte de modelado del objetivo general.

Para OE5, la ficha institucional establece como meta referencial un error porcentual absoluto medio (MAPE) inferior al 15–20 % en las métricas principales. Esta meta es ajustable después de los experimentos piloto. No constituye, por tanto, un compromiso cerrado sobre la precisión del modelo predictivo.

Leídos en conjunto, los cinco objetivos forman una cadena: el entorno de OE1 sostiene el benchmark de OE2, cuyas cargas producen las mediciones de OE3, que alimentan el análisis por fases de OE4 y, a través de él, el modelo predictivo de OE5. La dimensión de calidad que el objetivo general incluye se aborda mediante la comparación limitada entre BF16 y AWQ INT4 que precisa la Sección 1.6.1. Queda por justificar por qué esta cadena merece recorrerse, tarea que asume la Sección 1.5.

1.5 Justificación

La cadena de objetivos se justifica, en primer lugar, desde el punto de vista técnico. El análisis separado de prefill y decode permite explicar los compromisos entre latencia, throughput y memoria que las métricas agregadas solo registran. Williams et al. [10] proponen el modelo roofline, que relaciona el rendimiento alcanzable con los límites de cómputo y de ancho de banda de memoria del procesador. Este marco ofrece una referencia para situar cada fase respecto a los límites de la A100 y sustenta el análisis de cuellos de botella de OE4.

Desde el punto de vista operativo, el trabajo orienta la elección de configuraciones para los tres escenarios de uso presentados en la Sección 1.2. Según la ficha institucional, el chat interactivo prioriza un TTFT y un TPOT bajos; la consulta documental requiere un equilibrio entre longitud de entrada, latencia y memoria; y el procesamiento por lotes prioriza el throughput y la eficiencia energética. Una caracterización por fases permite relacionar cada una de esas prioridades con la fase que la gobierna.

La utilidad del trabajo se materializa en el prototipo Qwen-A100 Profiler, que la ficha institucional define como su producto tecnológico. Esta herramienta automatiza la ejecución de los experimentos, recolecta las métricas, almacena los resultados de forma estructurada y estima el TTFT y el TPOT mediante el modelo predictivo de OE5. El conjunto de datos experimental y los procedimientos de reproducción completan los entregables reutilizables, de modo que el estudio puede repetirse sin reconstruir la metodología.

Esos entregables definen a los beneficiarios del trabajo. El beneficiario directo es el proyecto LLM-CoDesign, que obtiene su primer caso de estudio y una metodología de caracterización aplicable a otras combinaciones de modelo, hardware y software. Los beneficiarios indirectos son los equipos técnicos que despliegan LLM en infraestructura propia y necesitan criterios cuantitativos para configurar el servicio.

La justificación exige, por último, que el trabajo sea factible. En el plano técnico, Qwen3-8B cabe en la memoria de una única A100, y las herramientas necesarias (vLLM, las herramientas de telemetría NVML y DCGM, y las bibliotecas de análisis de datos) son de libre acceso. El trabajo no requiere entrenar el modelo, desarrollar kernels de GPU ni modificar internamente vLLM. En el plano operativo, el plan de trabajo comprende doce semanas con una dedicación de 15 a 20 horas semanales, y la factibilidad depende de disponer de acceso controlado a la GPU durante las campañas de medición. Justificada la pregunta, falta precisar hasta dónde llega la respuesta, cometido de la Sección 1.6.

1.6 Alcance y delimitaciones

La respuesta a la pregunta central vale únicamente dentro de un dominio experimental definido. La Sección 1.6.1 precisa ese dominio y la Sección 1.6.2 enumera lo que queda fuera de él.

1.6.1 Alcance

La unidad de análisis es la configuración experimental $C = (M, W, H, S)$. En ella, M es el modelo Qwen3-8B; W, las características de la carga de trabajo; H, una GPU NVIDIA A100; y S, vLLM con sus parámetros de configuración. La variante exacta de la A100, de 40 GB u 80 GB, se documenta como parte del entorno experimental, porque su capacidad de memoria condiciona los niveles máximos de longitud de contexto y concurrencia alcanzables.

Sobre esa unidad se varían cuatro factores, que constituyen las variables experimentales. La cuantización se evalúa a través de la representación numérica, con dos niveles: BF16 y AWQ INT4. La longitud de entrada toma cuatro niveles, de 128, 512, 2048 y 8192 tokens; la longitud de salida, dos niveles, de 64 y 256 tokens; y la concurrencia, cuatro niveles, de 1, 4, 8 y 16 solicitudes simultáneas. La combinación de los niveles produce $2 \times 4 \times 2 \times 4 = 64$ configuraciones, y con cinco repeticiones por configuración la campaña comprende alrededor de 320 ejecuciones medidas. Las configuraciones se agrupan además en tres tipos de carga de trabajo: chat corto, consulta documental y contexto largo.

Las métricas registradas son el TTFT, el TPOT, el throughput, la latencia total de la solicitud, la memoria máxima utilizada, la utilización de la GPU, la potencia media y la energía estimada por token. La energía se presenta como una estimación basada en la telemetría interna de la GPU, obtenida mediante NVML y DCGM. No constituye una medición eléctrica calibrada externamente, salvedad que la ficha institucional exige hacer explícita.

El alcance incluye una comparación limitada de calidad entre BF16 y AWQ INT4 para identificar posibles degradaciones asociadas a la cuantización. Incluye también el prototipo Qwen-A100 Profiler en su versión de producto mínimo viable, con una recomendación básica de configuraciones para los tres escenarios de uso. Definido lo que el trabajo abarca, la Sección 1.6.2 fija lo que excluye.

1.6.2 Delimitaciones

Las exclusiones siguientes acotan el trabajo a un caso controlado e impiden que sus conclusiones se extiendan más allá de lo medido:

- Entrenamiento completo o fine-tuning del modelo.
- Comparación con otras familias de LLM.
- Evaluación de múltiples GPU y ejecución multi-GPU.
- Paralelismo tensorial o de pipeline.

- Comparación entre varios motores de inferencia.
- Modelos Mixture-of-Experts o multimodales.
- Speculative decoding.
- Longitudes de entrada superiores a 8192 tokens en la campaña principal.
- Predicción del comportamiento en otro hardware.
- Simulación de sistemas distribuidos.
- Desarrollo completo del asesor multiobjetivo de LLM-CoDesign.

Las conclusiones del trabajo serán válidas únicamente para la combinación de Qwen3-8B, NVIDIA A100 y vLLM. No se formularán afirmaciones de superioridad general sobre otras plataformas o modelos. Delimitado así el dominio de la respuesta, la Sección 1.7 expone los fundamentos teóricos que los capítulos siguientes requieren.

1.7 Fundamentación teórica

[Desarrollar los fundamentos teóricos, conceptos, modelos y tecnologías que sustentan el proyecto. Debe estar relacionado directamente con la solución propuesta y su implementación, aplicable a software, hardware o infraestructura.]

CAPÍTULO II

METODOLOGÍA

2.1 Diagnóstico de la Situación Actual

[Analizar el estado inicial del sistema, proceso o infraestructura antes de la implementación del proyecto. Describir cómo funciona actualmente y las principales limitaciones técnicas.]

2.2 Área de Estudio

Tabla 2: Área de estudio.

Dominio:	*[Indique el dominio académico o científico del proyecto.]*
Línea de investigación:	Diseño, desarrollo e integración de sistemas basados en arquitecturas web, móviles y sistemas embebidos
Sublínea de investigación:	Optimización del rendimiento.
Campo:	Tecnologías de la información y la comunicación (TIC)
Área:	*[Defina el área técnica específica del proyecto.]*
Aspectos:	*[Describa los procesos, tecnologías o problemas que serán analizados.]*
Objeto de estudio:	*[Indique la organización, sistema, proceso o infraestructura objeto del estudio.]*
Periodo de estudio:	*[Indique las fechas o el intervalo de tiempo que comprende el estudio.]*

2.3 Metodología de Desarrollo o Implementación

[Describir y justificar la metodología empleada según la naturaleza del proyecto, indicando las fases y actividades a ejecutar. Por ejemplo, metodologías ágiles o tradicionales para software, o modelos de diseño, implementación o despliegue para hardware o infraestructura.]

2.4 Técnicas e Instrumentos de Recolección de Información

[Describir las técnicas utilizadas para obtener información durante el diagnóstico y la validación del proyecto, tales como entrevistas, encuestas u observación directa.]

2.5 Requerimientos del Proyecto

2.5.1 Requerimientos Funcionales

[Enumerar y describir los requerimientos funcionales que la solución debe cumplir.]

2.5.2 Requerimientos No Funcionales

[Describir requerimientos relacionados con seguridad, rendimiento, disponibilidad, escalabilidad u otros aplicables.]

2.6 Diseño de la Solución

[Describir el diseño técnico de la solución propuesta. Puede incluir arquitectura del sistema o infraestructura, diagramas lógicos y físicos, modelos de datos, componentes o configuraciones técnicas.]

2.7 Justificación Técnica

[Argumentar técnicamente la selección de tecnologías, herramientas, plataformas, equipos o servicios utilizados en el proyecto.]

La justificación técnica parte de las tecnologías que fija el caso de estudio: el motor de inferencia vLLM, que gestiona el KV cache mediante PagedAttention y planifica la ejecución a nivel de iteración [4]; el modelo Qwen3-8B [2]; y la GPU

NVIDIA A100 [3].

2.7.1 Ejemplo de Imagen 1

Ejemplo de referencia en el cuerpo del texto: la Imagen 1 puede usarse para documentar evidencia técnica del entorno, componentes o interfaz.



Imagen 1: Ejemplo de imagen para documentación visual de infraestructura o evidencias.

Elaborado por: Estudiante
Fuente: Captura referencial del proyecto

2.7.2 Ejemplo de Ecuación 1

Otro ejemplo de referencia: la Ecuación 1 muestra un cálculo típico para validación de disponibilidad del servicio.

$$Disponibilidad\ (\%) = \frac{T_{operativo}}{T_{total}} \times 100 \tag{1}$$

CAPÍTULO III

IMPLEMENTACIÓN, PRUEBAS Y RESULTADOS

3.1 Descripción de la Solución

[Presentar detalladamente la solución implementada, indicando los componentes desarrollados, configurados o integrados.]

3.2 Proceso de Implementación

[Describir paso a paso la ejecución técnica del proyecto, incluyendo desarrollo o configuración, integración de componentes y despliegue en un entorno real o controlado.]

3.3 Pruebas y Validación

[Describir las pruebas realizadas para validar el funcionamiento de la solución, tales como pruebas funcionales, integrales, de seguridad, rendimiento o conectividad, según aplique.]

3.4 Resultados

[Presentar los resultados obtenidos durante la validación, comparando la situación inicial con la posterior a la implementación. Incluir métricas, indicadores o evidencias.]

3.4.1 Ejemplo de Figura 2

Como ejemplo de mención con referencia: en la Figura 1 se observa una salida visual que puede emplearse para documentar arquitectura o resultados.
Elaborado por: Estudiante
Fuente: Elaboración propia



Figura 1: Ejemplo de figura para presentar resultados o arquitectura implementada.

3.4.2 Ejemplo de Tabla 2

Como ejemplo de mención con referencia: la Tabla 3 muestra una comparación antes/después para análisis cuantitativo.

Tabla 3: Ejemplo de tabla comparativa antes y después de la implementación.

Métrica	Antes	Después
Tiempo de respuesta (ms)	850	320
Disponibilidad (%)	96.2	99.3
Tasa de error (%)	4.8	1.2

Elaborado por: Estudiante
Fuente: Resultados de pruebas funcionales

3.4.3 Ejemplo de Imagen 2

Como ejemplo de referencia narrativa: la Imagen 2 permite incorporar evidencias de interfaz o panel operativo.
Elaborado por: Estudiante
Fuente: Evidencia del entorno de prueba

3.4.4 Ejemplo de Ecuación 2

Ejemplo de referencia a ecuación: la Ecuación 2 calcula el porcentaje de mejora de una métrica respecto al valor inicial.



Imagen 2: Ejemplo de imagen para evidenciar interfaz, panel o consola de operación.

$$Mejora\ (\%) = \frac{Valor_{antes} - Valor_{despues}}{Valor_{antes}} \times 100 \tag{2}$$

3.4.5 Ejemplos de Referencias Cruzadas

Ejemplo 1 de referencia cruzada entre capítulos: el análisis base se desarrolla en el Capítulo I, mientras que el enfoque metodológico se detalla en el Capítulo II.

Ejemplo 2 de referencia cruzada entre elementos: para relacionar diseño y validación se puede enlazar el apartado *Diseño de la Solución* (Capítulo II) con la Tabla 3, la Imagen 1 y la Ecuación 1.

3.5 Cronograma y Presupuesto

[Presentar la planificación de las fases de ejecución del proyecto y el detalle de los costos asociados a hardware, licencias, servicios u otros recursos.]

CAPÍTULO IV

CONCLUSIONES Y RECOMENDACIONES

4.1 Conclusiones

[Redactar conclusiones basadas en los resultados obtenidos y en el cumplimiento de los objetivos planteados.]

4.2 Recomendaciones

[Proponer mejoras, optimizaciones o ampliaciones del proyecto, fundamentadas en el análisis realizado.]

4.3 Trabajos Futuros

[Plantear líneas de desarrollo o investigación que puedan derivarse del proyecto implementado.]

[Incluya todas las referencias bibliográficas utilizadas en el documento. Deben estar citadas según normas IEEE y gestionadas mediante un gestor bibliográfico como Mendeley o Zotero.]

BIBLIOGRAFÍA

[1] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, 2024. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

[2] Qwen Team, "Qwen3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[3] NVIDIA Corporation, "NVIDIA A100 tensor core GPU datasheet," https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-1758950-r4-web.pdf, 2021, consultado el 16 de octubre de 2026.

[4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Koblenz, Germany: ACM, 2023, arXiv:2309.06180.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. Curran Associates, Inc., 2017, arXiv:1706.03762.

[6] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. Santa Clara, CA: USENIX Association, 2024. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/agrawal

[7] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, 2022, pp. 521–538. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/yu

[8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-Awareness," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, arXiv:2205.14135.

[9] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proceedings of Machine Learning and Systems 6 (MLSys 2024)*, Santa Clara, CA, 2024, best Paper Award. arXiv:2306.00978.

[10] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

GLOSARIO

[Defina los términos técnicos relevantes utilizados en el documento.]

Término o sigla 1 *[Definición técnica breve y clara.]*

Término o sigla 2 *[Definición técnica breve y clara.]*

ANEXO A

TÍTULO DEL ANEXO A

[Incluya una descripción breve del contenido del anexo. Ejemplo: documentación técnica, diagramas, configuraciones, evidencias o instrumentos aplicados.]

- *[Documentación técnica.]*
- *[Manuales de usuario y administración.]*
- *[Evidencias de pruebas.]*
- *[Autorizaciones y validaciones de la entidad receptora.]*
- *[Repositorio de versionamiento (enlace).]*
- *[Formatos de entrevistas y encuestas aplicadas.]*